Marlita C. Acibron
Micah Mae C. Leguis

BSIT 2 E
November 10, 2026

Campus Cafe Ordering System Using Object-Oriented Programming

Concept Explanation

1. Classes and Objects We have seven classes and create objects in main() so each real-world café item and customer has its own data and actions.

2. **Encapsulation** We use private fields plus getters and validated setters so data cannot be changed directly from outside the class.

3. **Inheritance** Beverage, Pastry and Meal extend MenuItem, reusing common code instead of writing it again.

4. **Polymorphism** Each subclass gives its own version of computePrice() so different items calculate their price in their own way.

5. **Abstraction The abstract** MenuItem class defines the required computePrice() method without writing its full logic.

6. **Method Overloading** addPoints() in Customer and computeTotal() in Order have same names but different parameters for different inputs.

7. **Method Overriding** With @Override, subclasses rewrite inherited computePrice() and getDescription() to match their own rules.

Program Output

The program demonstrates a campus cafe ordering system. It displays customers, food and beverage items, prices, and order details. The first customer, Ana Reyes, is a student. Her order includes an Iced Latte, Ensaymada, and Chicken Adobo. The program also demonstrates what happens when an additional item is not accepted because the maximum number of items has been reached. The second customer, Ben Cruz, orders Hot Chocolate and Beef Tapa. The program calculates the order total and demonstrates the applicable discounts or vouchers.

Challenges Encountered and Solutions

While creating the program, We encountered challenges in understanding how classes, objects, inheritance, and polymorphism work together. we addressed these by organizing the program into separate Java files and using inheritance to share common properties and methods. Another challenge was calculating prices based on selected sizes, additional items, and discounts. we used conditional statements and the appropriate methods to handle these calculations. We also reviewed the code, corrected errors, and tested the program again to make sure it worked properly.

Conclusion

Through this activity, We learned how Object-Oriented Programming concepts are applied in a real-world situation. we understood how classes and objects work together, how inheritance reduces repeated code, and how polymorphism allows different objects to perform their own actions. This activity improved my Java programming skills, problem-solving abilities, and understanding of how to organize a program using multiple classes. I realized that practicing and correcting mistakes are important parts of learning programming.